1-Two Sum

Given an array of Integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

You may assume that each input would have exactly one solution and you may not use the same element twice. You can return the answer in any order.

Example

Given : `nums = [2,7,11,15], target = 13`

Explanation : Numbers 2 and 11 adds up to the target 13

Number 2 is at `index-0`

Number 11 is at `index-2`

`output = [0,2]`

Code Implementation - Python

```python
**********************************************************************
* This function will get an input array of elements, the value
* target      : 13
* numbers     : [2, 7, 11, 15]
* difference  : [11, 6, 2, -2]
**********************************************************************
def twoSum(nums, target):
    # initialise the hashmap
    hashMap = {}

    # take the index and number from the list
    for index, num in enumerate(nums):
        # i : 0 --> diff = 13 - nums[0] = 13 - 2 = 11
        # i : 1 --> diff = 13 - nums[1] = 13 - 7 = 6
        # i : 2 --> diff = 13 - nums[2] = 13 - 11 = 2
        # i : 3 --> diff = 13 - nums[3] = 13 - 15 = -2
        difference = target - num

        # When the difference exists in hashMap (as key)
        if difference in hashMap:
            # extract the value from the hashMap by using key as "difference"
            firstIndex = hashMap[difference]
            # assign the current index as second index
            secondIndex = index
            # return the two indices
            return [firstIndex, secondIndex]

        hashMap[num] = index

    return [-1,-1]
```

Listing 1: *Two Sum-Python*

Code Implementation - Java

```java
/**********************************************************************
* This function will get an input array of elements, the value
* target      : 13
* indices     : [0, 1, 2, 3]
* numbers     : [2, 7, 11, 15]
* difference  : [11, 6, 2, -2]
**********************************************************************/
public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> numbersMap = new HashMap<>();
    for(int i = 0; i < nums.length; i++) {
        // i : 0 --> diff = 13 - nums[0] = 13 - 2 = 11
        // i : 1 --> diff = 13 - nums[1] = 13 - 7 = 6
        // i : 2 --> diff = 13 - nums[2] = 13 - 11 = 2
        // i : 3 --> diff = 13 - nums[3] = 13 - 15 = -2
        int difference = target - nums[i];

        // If the difference already exist in map, return the index along with current index
        if(numbersMap.containsKey(difference)) {
            return new int[]{numbersMap.get(difference), i};
        }
        else {
            numbersMap.put(nums[i], i);
        }
    }
    throw new RuntimeException("No numbers pair up to : " + target);
}
```

Listing 2: *Two Sum - Java*

Description - Example 1

Let us debug the code. For each element of the array we have to find its difference from the target. So we note it down as follows. In our case target(T) is 13.

Target (T) = 13				
Element (E)	2	7	11	15
Diff ($D = T - E$)	11	6	2	-2

Array Elements and Difference

A Hashmap is initialised to store the Element and its index to recover the solution later.

```
Map<Integer, Integer> numbersMap = new HashMap<>();
```

We start iterating through all the numbers and the numbersMap is empty.

```
for(int i = 0; i < nums.length; i++) {
| ......
}
```

Iteration-0 : At $i \longrightarrow 0$, for element $i_0 \rightarrow 2$ difference is $T - E \longrightarrow 13 - 2 = 11$. We look into HashMap and if it does not exists just add the element (2) and its index (0) as key value pair.

```
if(numbersMap.containsKey(difference)) {
| ........
} else {
// {e : index }
// {2 : 0}
numbersMap.put(nums[i], i);
}
```

Iteration-1 : At $i \longrightarrow 1$, for element $i_1 \rightarrow 7$ difference is $T - E \longrightarrow 13 - 7 = 6$. We check the HashMap if 6 exists. Since it is not, we just add it again.

```
| if(numbersMap.containsKey(difference)) {
| | ........
| } else {
| | // {e : index }
| | // {2 : 0}
| | // {7 : 1}
| | numbersMap.put(nums[i], i);
| }
```

Iteration-2 : At $i \longrightarrow 2$, for element $i_2 \rightarrow 11$ difference is $T - E \longrightarrow 13 - 11 = 2$. This difference value 2 exists in HashMap already. So we enter the if condition and look for the index by giving the element as key to get the value(index) from the map.

```
/*
{{el, index}
 {2, 0}
 {7, 1}
 {11, 2} }
*/
// difference = 2 ⟶ look up element
if(numbersMap.containsKey(difference)) {
| // element(value) = numbersMap.get(difference)
| // element(value) = numbersMap.get(2)
| // element(value) = 0(0th index)
| // ----------------------------
| // indexxx : | 0 | 1 | 2 |
| // ----------------------------
| // element : | 2 | 7 | 11 |
| // ----------------------------
| int currentElementIndex = i; // currentElementIndex ⟶ 2
|
| int complimentElementIndex = numbersMap.get(difference) // complimentElementIndex ⟶ 0
|
| return new int[]{complimentElementIndex, currentElementIndex};
}
```

Our map already contains the following $< key, value >$ pairs $i.e., \longrightarrow \{2,0\},\{7,1\},\{11,2\}$. The iteration i_2 has the difference 2 and we know this difference exists in the map as one of the key $i.e., \{2,0\}$. So we get the value of the map and it produces the result we need $i.e.$, index $\longrightarrow 0$. Thus we got the answer `current_index = 2` and `target_index = 0`.

Thus we get an integer array $[2,0]$ as output.